

Prof. Dr. Matthias Wählisch
Faculty of Computer Science | Chair of Distributed and Networked Systems

Advanced Computer Networks

DNS and Email

Motivation

OUTAGE ANALYSIS Cloudflare Outage Analysis: November 18, 2025 [READ BLOG →](#)

CISCO
ThousandEyes 

PRODUCT SOLUTIONS LEARN ABOUT  EN US  BLOG [REQUEST DEMO](#) [SUBSCRIBE](#)

OUTAGE ANALYSES

Microsoft Azure Front Door Outage Analysis: October 29, 2025

By [Internet Research Team](#) | October 29, 2025 | 16 min read

The New York Times

Oct 19-20, 2025

Amazon Outage Forces Hundreds of Websites Offline for Hours

Amazon Web Services, a major provider of cloud services, cited a problem at its data center in Northern Virginia. The outage highlighted the fragility of global internet infrastructure.

Motivation

Automated programs called Enactors (“agents” if you will, minus the ML) speed up updating of DNS

1. Slow DNS Enactor reads the current plan’s timestamp. See it is plan_old
2. Starts applying plan_new
3. ...Meanwhile a fast DNS enactor creates plan_newest...
4. ...Fast enactor replaces everything with plan_newest...
5. ...Fast enactor **deletes plan_new**, because it is older than plan_newest
6. Slow enactor **overwrites plan_newest with plan_new** (now empty due to deletion in step 5), because it **checked plan freshness only in step 1**

The New York Times

Oct 19-20, 2025

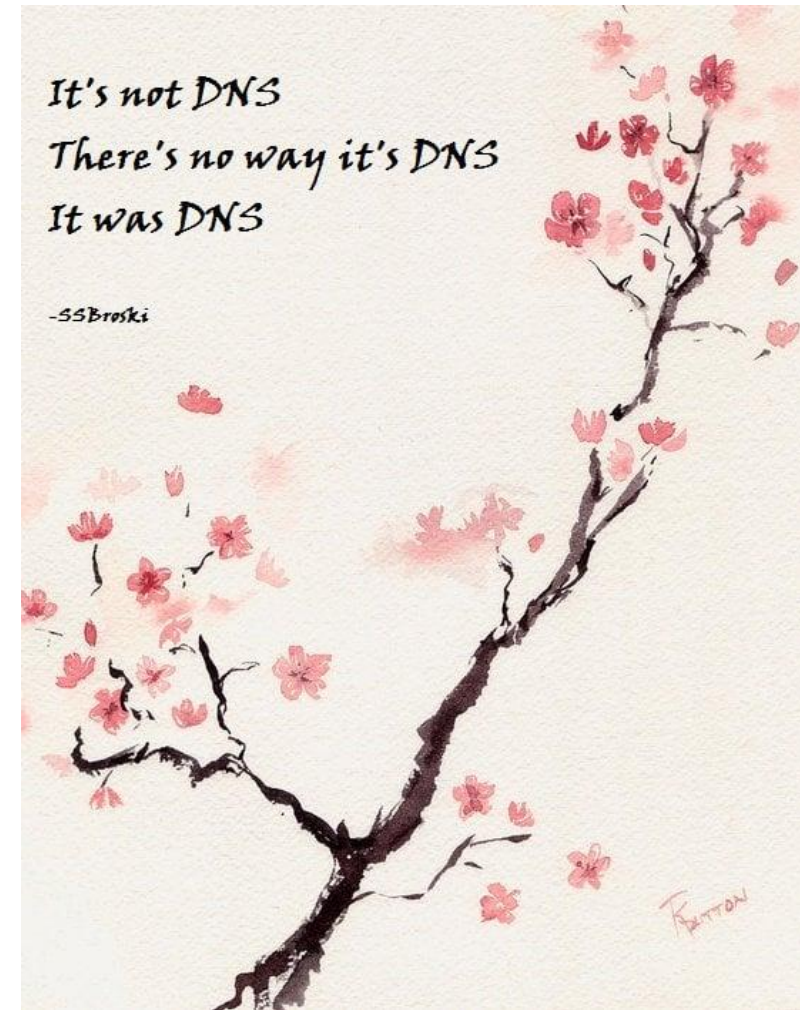
Amazon Outage Forces Hundreds of Websites Offline for Hours

Amazon Web Services, a major provider of cloud services, cited a problem at its data center in Northern Virginia. The outage highlighted the fragility of global internet infrastructure.

Motivation

In both cases, the DNS was affected but the DNS was not the root cause ;). It still shows that an intact DNS is utmost important.

In AWS, root cause was assuming your program will “run fast enough” and that there will be no concurrency and thus no need for concurrency control.



DNS(SEC)

Domain Name System

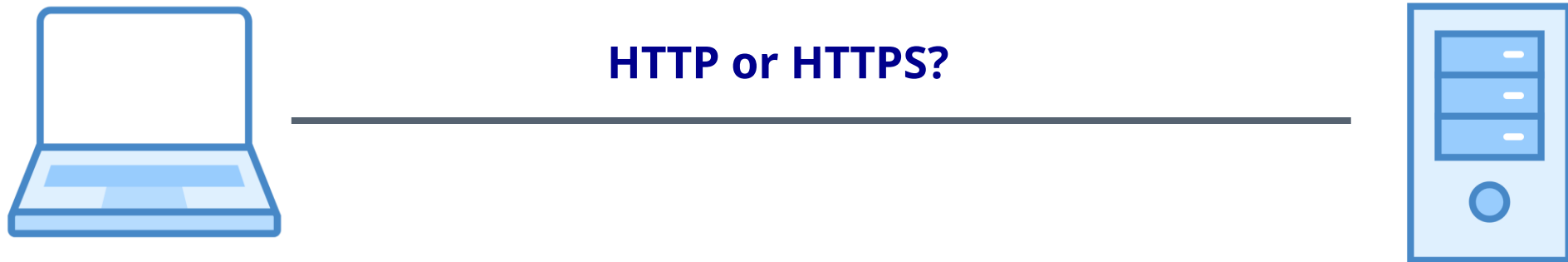
Map a name to resource records to values.

Publish the resource scalable and distributed.

Achieve scalability by delegation and caching.

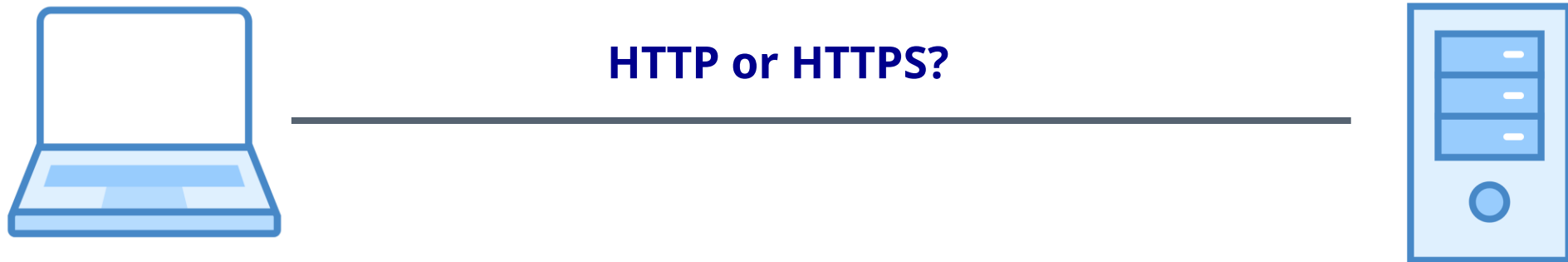
Is the `A` or `AAAA` the only record that is needed?

Is the `A` or `AAAA` the only record that is needed?
Use case negotiating HTTP security and performance.



Is the `A` or `AAAA` the only record that is needed?

Use case Negotiating HTTP security and performance.



Starting with HTTP, use HTTP redirect to `https://` costs time.

Service bindings via DNS

The generic DNS record “SVCB” can be instantiated into records specific to different protocols, e.g., HTTPS.

Goal: Provide all required information to make a connection to a service with fewer DNS queries, so less round-trip time for application protocol connections.

Service bindings via DNS

The generic DNS record “SVCB” can be instantiated into records specific to different protocols, e.g., HTTPS.

Goal: Provide all required information to make a connection to a service with fewer DNS queries, so less round-trip time for application protocol connections.

Examples:

```
$ dig www.google.de HTTPS
www.google.de.      21600      IN          HTTPS      1 . alpn="h2,h3"

$ dig www.cloudflare.com HTTPS
www.cloudflare.com.  295        IN          HTTPS      1 . alpn="h3,h2"
ipv4hint=104.16.123.96,104.16.124.96
ipv6hint=2606:4700::6810:7b60,2606:4700::6810:7c60
```

Service bindings via DNS

Further advantages?

Examples:

```
$ dig www.google.de HTTPS
www.google.de.      21600      IN          HTTPS      1 . alpn="h2,h3"

$ dig www.cloudflare.com HTTPS
www.cloudflare.com.  295        IN          HTTPS      1 . alpn="h3,h2"
ipv4hint=104.16.123.96,104.16.124.96
ipv6hint=2606:4700::6810:7b60,2606:4700::6810:7c60
```

Service bindings via DNS

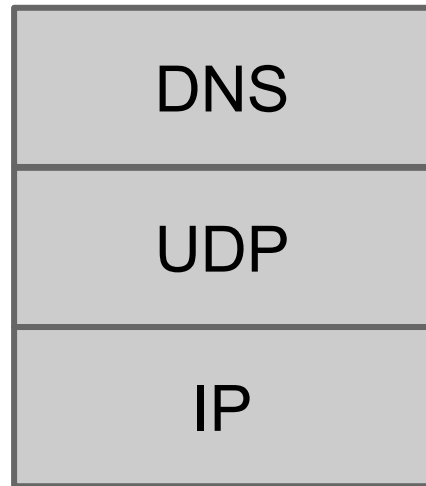
Advantages:

Fewer round trip times because the clients does not need to re-establish TCP+TLS connection.

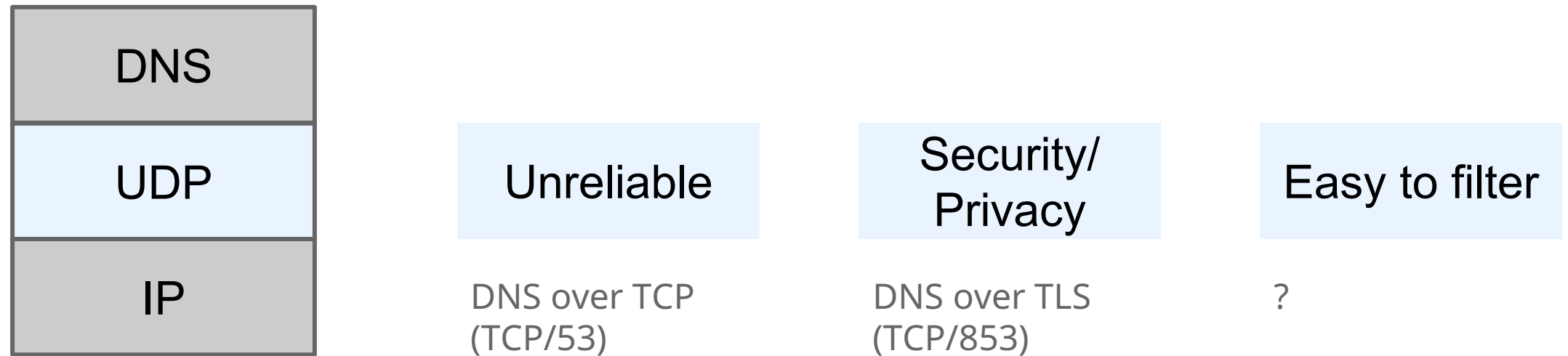
Increased security because an attacker cannot redirect the client misusing (unprotected) HTTP.

Increased privacy because the initial request is not leaked to the network if you use a secure DNS channel.

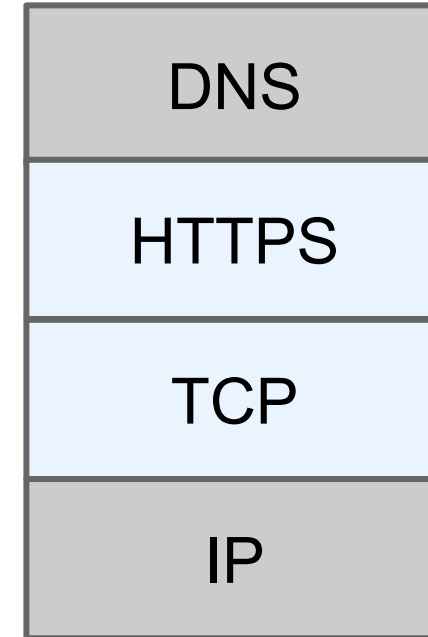
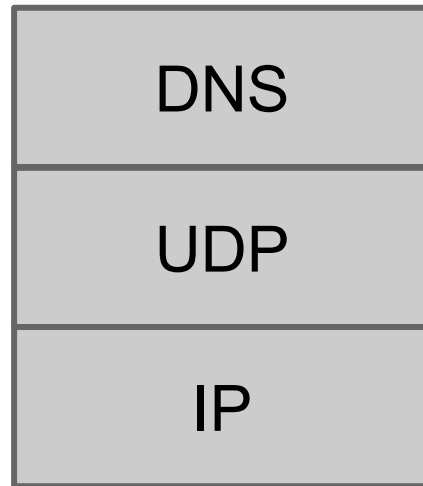
The DNS transport you know



Some problems can be solved



DNS over HTTPS (DoH) [RFC 8484]



Selection of DoH server

DoH Client is configured with a URI template

e.g., `https://dnsserver.example.net/dns-query{?dns}`

Configured out of band

By selecting the URI template, the client chooses the DoH server

A server may support multiple URI templates

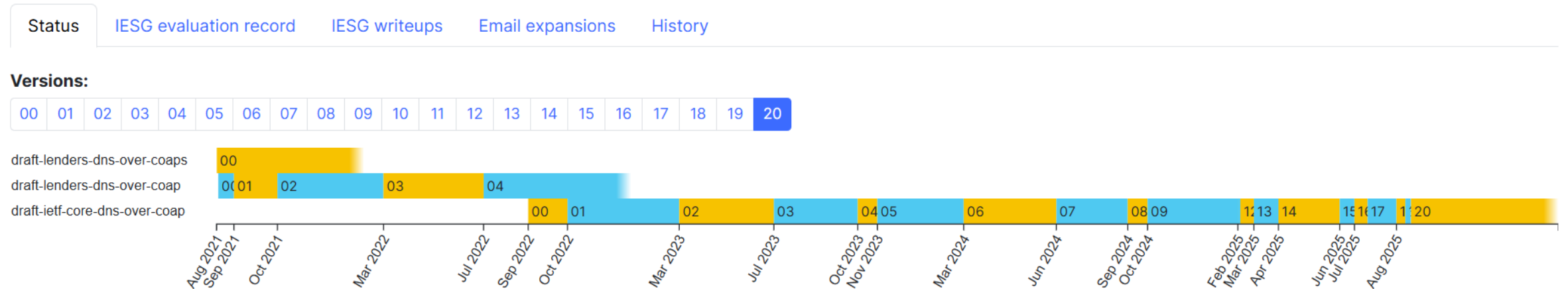
In constrained networks, we propose DNS over CoAP

<https://datatracker.ietf.org/doc/draft-ietf-core-dns-over-coap/>

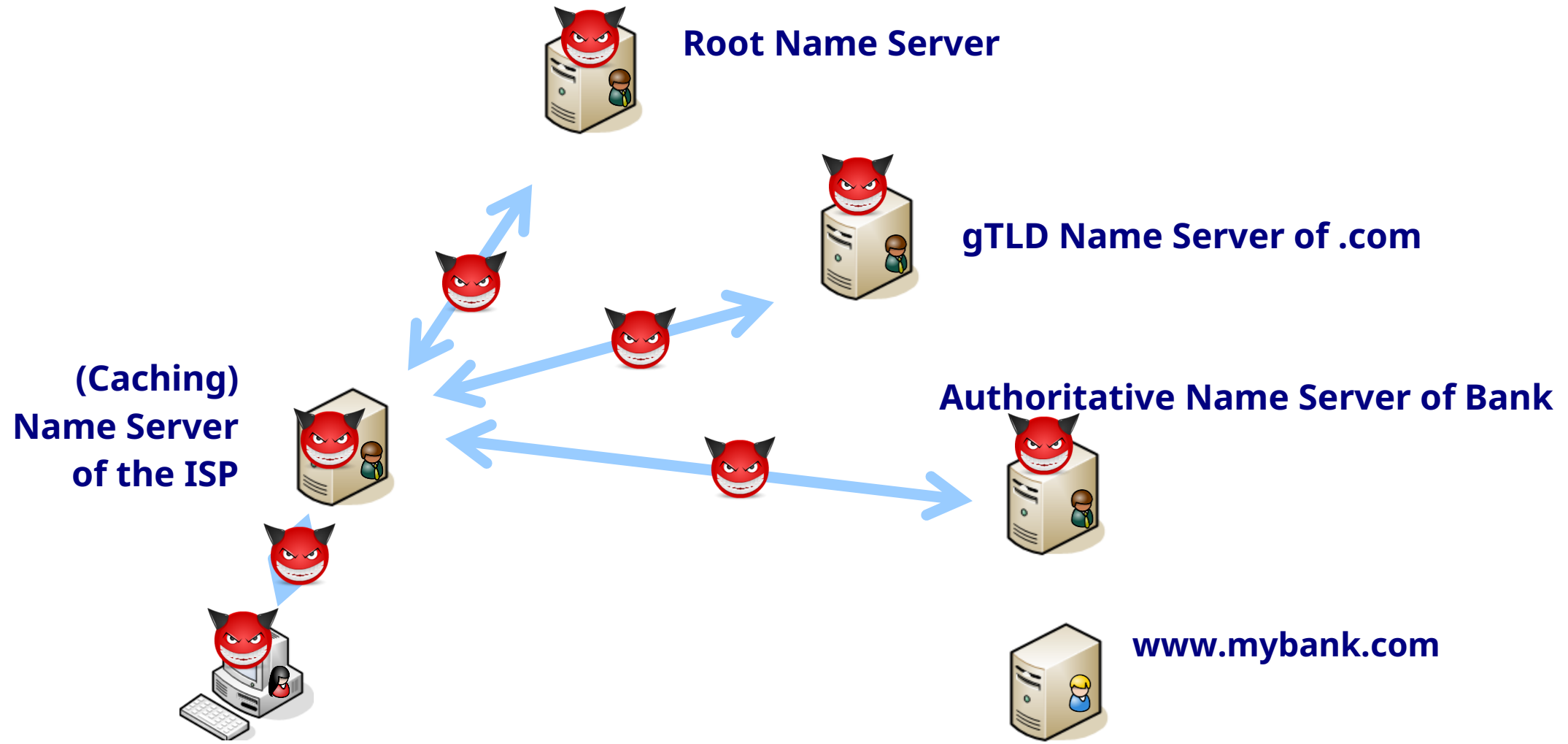
In constrained networks, we propose DNS over CoAP

<https://datatracker.ietf.org/doc/draft-ietf-core-dns-over-coap/>

DNS over CoAP (DoC) draft-ietf-core-dns-over-coap-20



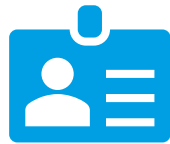
Attack surface of DNS



There are different security objectives



Confidentiality to
ensure nondisclosure
of data



Authenticity to allow
for the verification of
identity

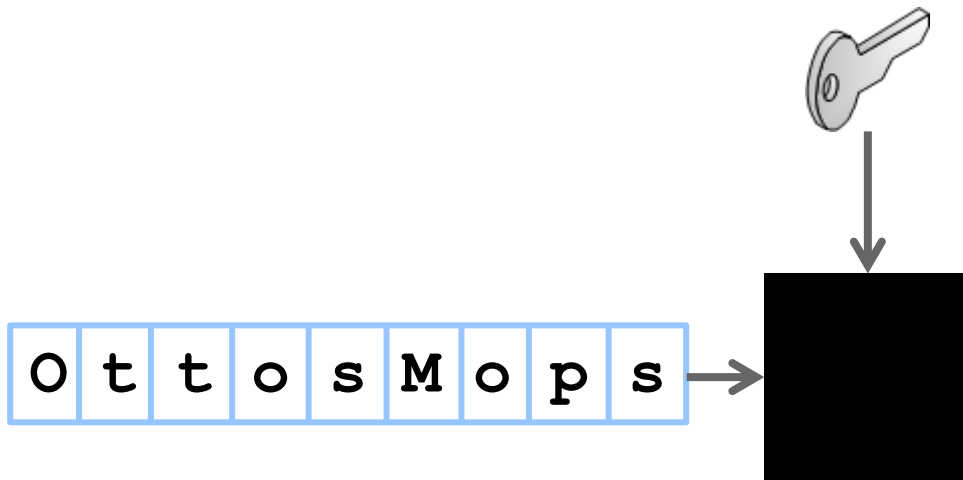


Integrity to ensure
accuracy and
completeness of data



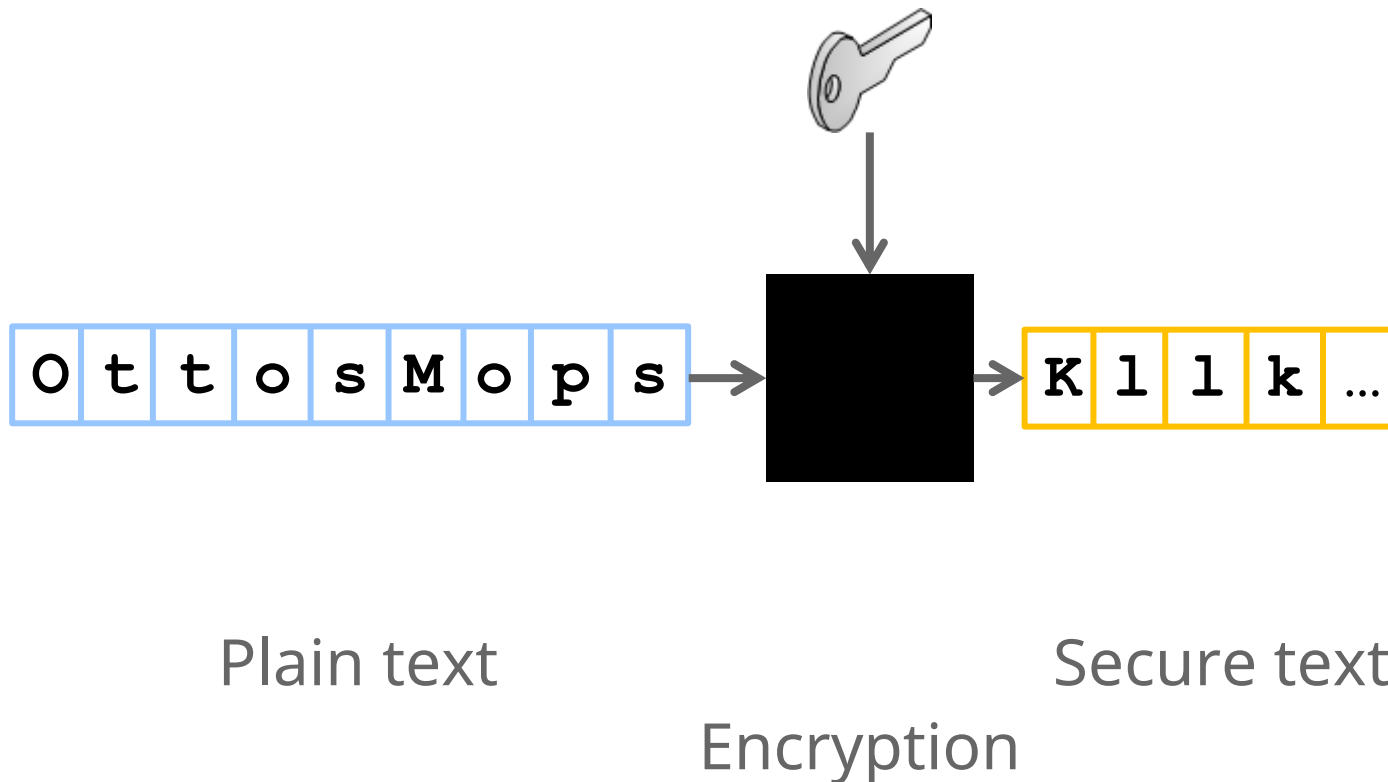
+++

You achieve many security goals by encryption

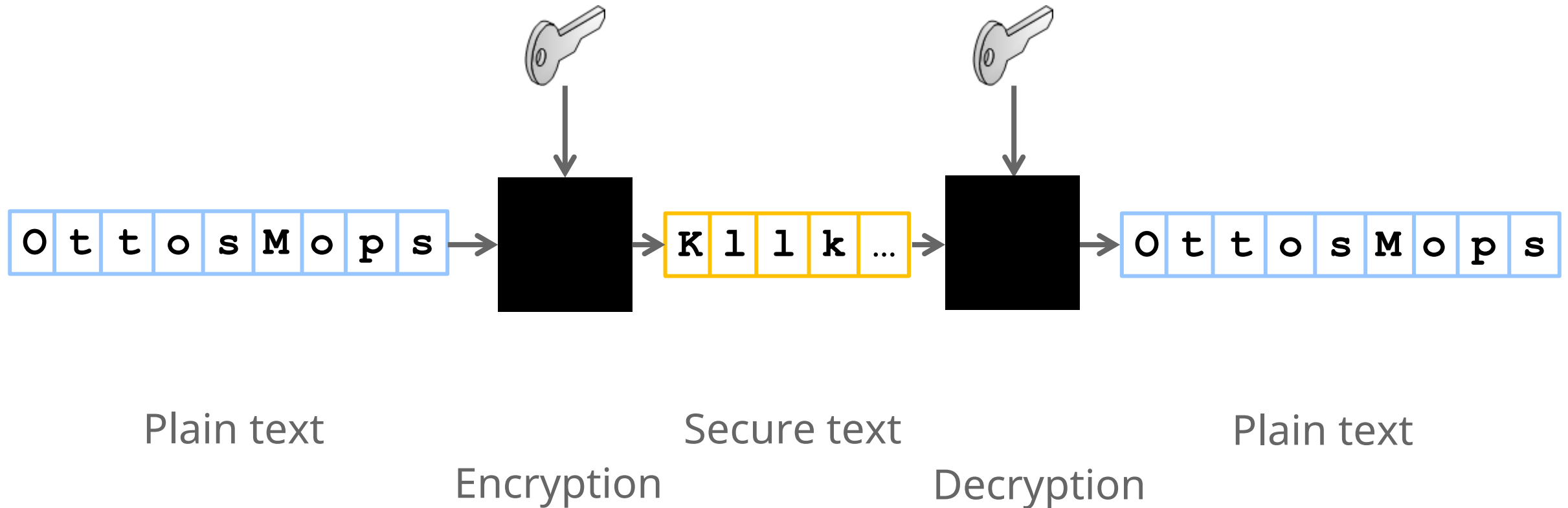


Plain text

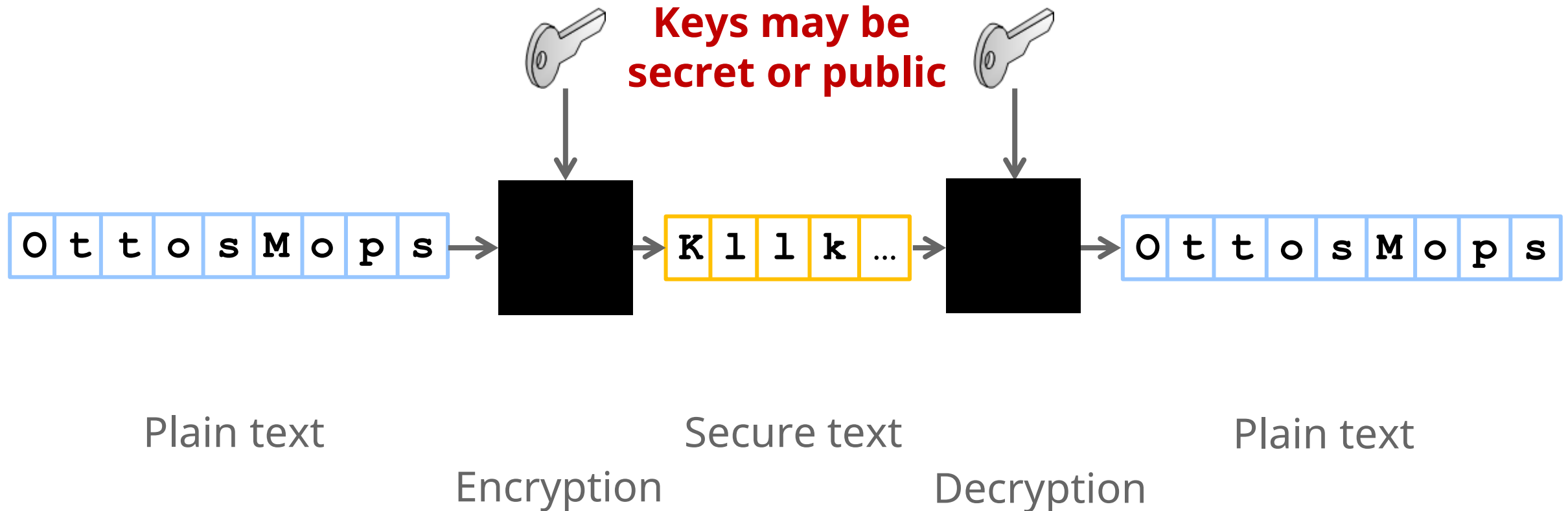
You achieve many security goals by encryption



You achieve many security goals by encryption



You achieve many security goals by encryption



Symmetric vs. asymmetric cryptography

Symmetric cryptography

Same key is used for encryption and decryption

Key is **always private**

Very efficient calculation

Asymmetric cryptography

Different keys are used for encryption and decryption

Two types of keys: **Private** key and **public** key

Complex numerical calculation

Usually used to exchange private keys for symmetric cryptography

Symmetric vs. asymmetric cryptography

Symmetric
cryptography

Confidentiality by encryption

Authentication by challenge
response scheme

Asymmetric
cryptography

?

Private public key encryption

	Public key	Private key
Confidentiality	Encryption of message	Decryption of message
Authenticity	Decryption of message	Encryption of message
Security goal	Application of keys	

One challenge in public private key crypto is to obtain **trust** in public key

How do you know that the **public key is valid**?

I generate a key pair (K_{pub} , K_{priv}) and claim that public key K_{pub} belongs to X. Then, I sign (cypher) a message with K_{priv} . When you verify the message and if you trust my original statement, you conclude that the message belongs to X. That's a wrong conclusion.

Problem: You need to trust the public key.

Public Key Infrastructure (PKI) manages that a public key belongs to entities such as persons or organizations.

When you trust the PKI, you trust all public keys managed by the PKI.

DNS challenges you want to solve

DNS resolver can lie about the DNS answer

DNS client cannot verify the correctness of the answer

DNSSEC Design Objectives [Current DNSSEC specs: RFCs 4033, 4034, 4035 (orig. 2535)]

Goals: Provide **integrity** to prevent spoofing and poisoning.

Approach:

Authenticating messages of name servers

Check that no one on the way changed the DNS content

Authenticating resource records

Check that data comes originally from authoritative DNS server

Proof of non-existence (prevent DoS against names)

Non-Goals: Confidentiality by hiding DNS data or requests, Authorization of requests or requestors, Protection against DDoS attacks (via traffic amplification)

DNSSEC Fundamentals

DNSSEC is based on public private key cryptography,
and introduces new resource records

RRSIG

contains cryptographic signature

DNSKEY

contains public signing key

DS

contains hash of the DNSKEY

NSEC/NSEC3

for explicit denial-of-existence of DNS records

CDNSKEY/CDS

for a child zone requesting updates to DS
record(s) in the parent zone

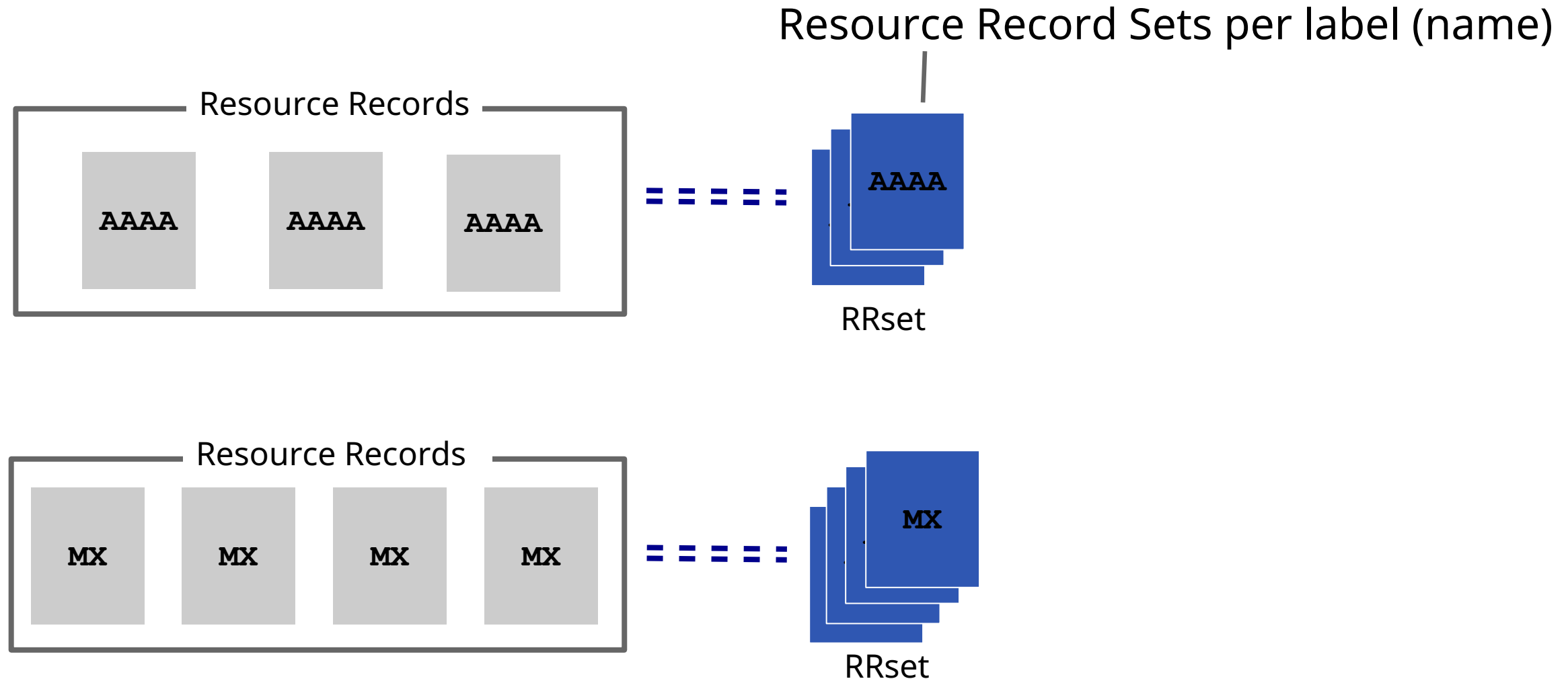
Secure a zone

Secure zone
delegation

Secure a zone

Secure zone
delegation

DNSSEC does not sign resource records individually



DNSSEC signs Resource Record Sets

RRset are all the RRs that have an identical **name**, **class** and **type**, e.g.,

```
www.opendnssec.se. 7200 IN A 185.49.141.14  
www.opendnssec.se. 7200 IN A 185.50.0.1
```

DNSKEY resource record that stores public key

```
www.opendnssec.se. 3600 IN DNSKEY 256 3 5  
AQOvhvXXU61Pr8sCwELcqqq1g4JJCALG4C9EtraBKVd+vG  
IF/unwigfLOAO3nHp/cgGrG6gJYe8OWKYNgq3kDChN
```

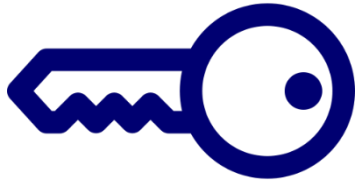
Public key

RRSIG resource record that contains signature of the RRset

```
www.opendnssec.se. 3600 IN RRSIG A 5 2 3600  
20050611144523 20050511144523 3112 opendnssec.se.  
VJ+8ijXvbrTLeoAiEk/qMrdudRnYZM1VlqhNvhYuAcYKe2X/jqYfMfj  
fSUrmhPo+0/GOZj66DJubZPmNSYXw==
```

Meta data +
Signature

DNSSEC distinguishes between two types of key pairs



Key-Signing-Key

KSK

To validate the ZSK

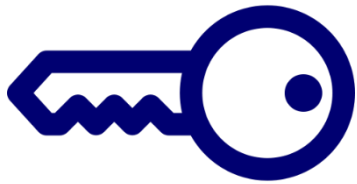


Zone-Signing Key

ZSK

To sign RRsets in the zone

DNSSEC distinguishes between two types of key pairs



Key-Signing-Key

KSK

To validate the ZSK

KSK operates in the same way as
ZSK secured the resource records
(1) Signs ZSK, which is stored in DNSKEY
(2) Creates RRSIG for the DNSKEY



Zone-Signing Key

ZSK

To sign RRsets in the zone

Secure a zone

Secure zone
delegation

DNSSEC applies a chain of trust to implement security

Start with a **root zone key**, the highest possible key in the DNS tree

In 2010, the first root full production DNSSEC root zone was signed

Then following cryptographic pointers to lower zones

In November 2011, more than 25% of top-level domains are signed with DNSSEC;

In February 2018, 91% (http://stats.research.icann.org/dns/tld_report/)

Each pointer is validated with the previous validated zone key

Using this mechanism only the root key is needed to validate *all* DNSSEC keys on the Internet

With these DNSSEC keys the DNS data in each zone can then be validated

Consequences of chain of trust

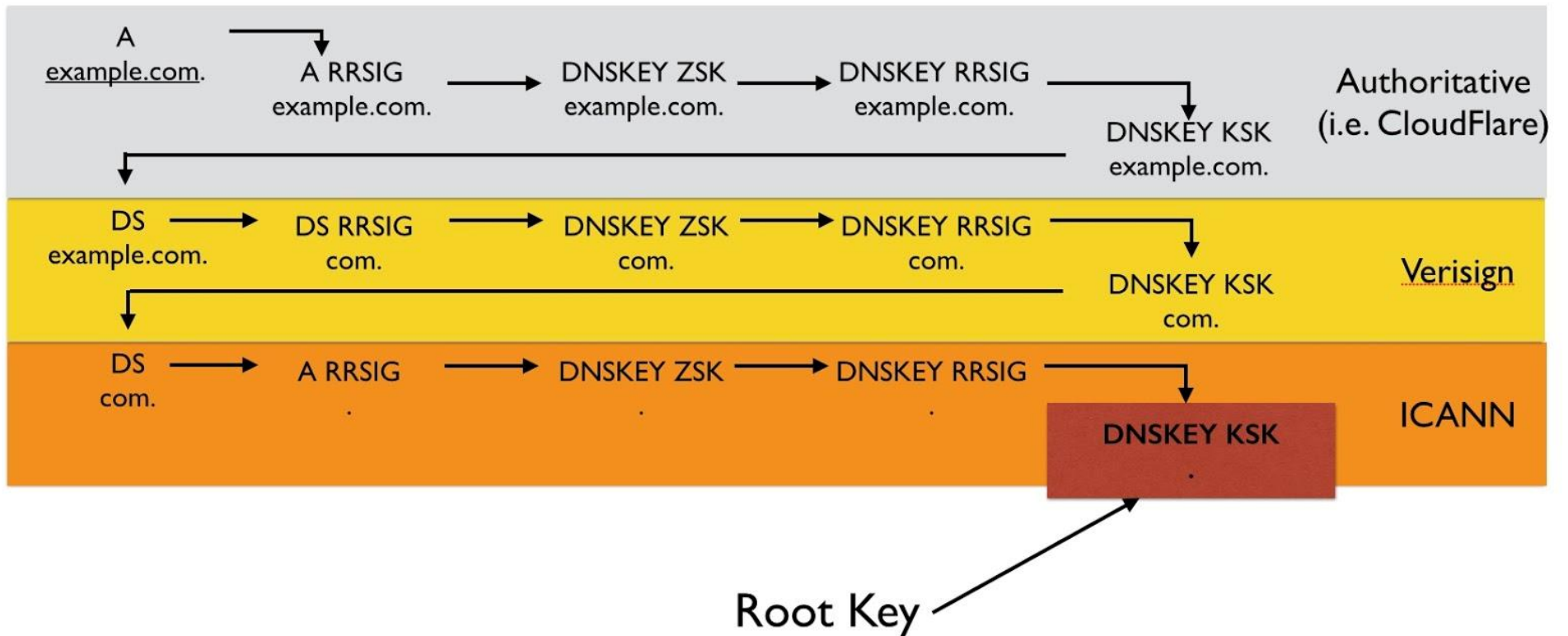
A root key must be anchored in the resolver at your local computer or nameserver

Only when a resolver knows and trusts a zone key it can validate the signatures belonging to that zone

Because of the chain of trust, a resolver has to carry *only the root key (or a few zone keys)* to be able to validate DNSSEC data on the Internet.

DNSSEC Trust Chain: Example

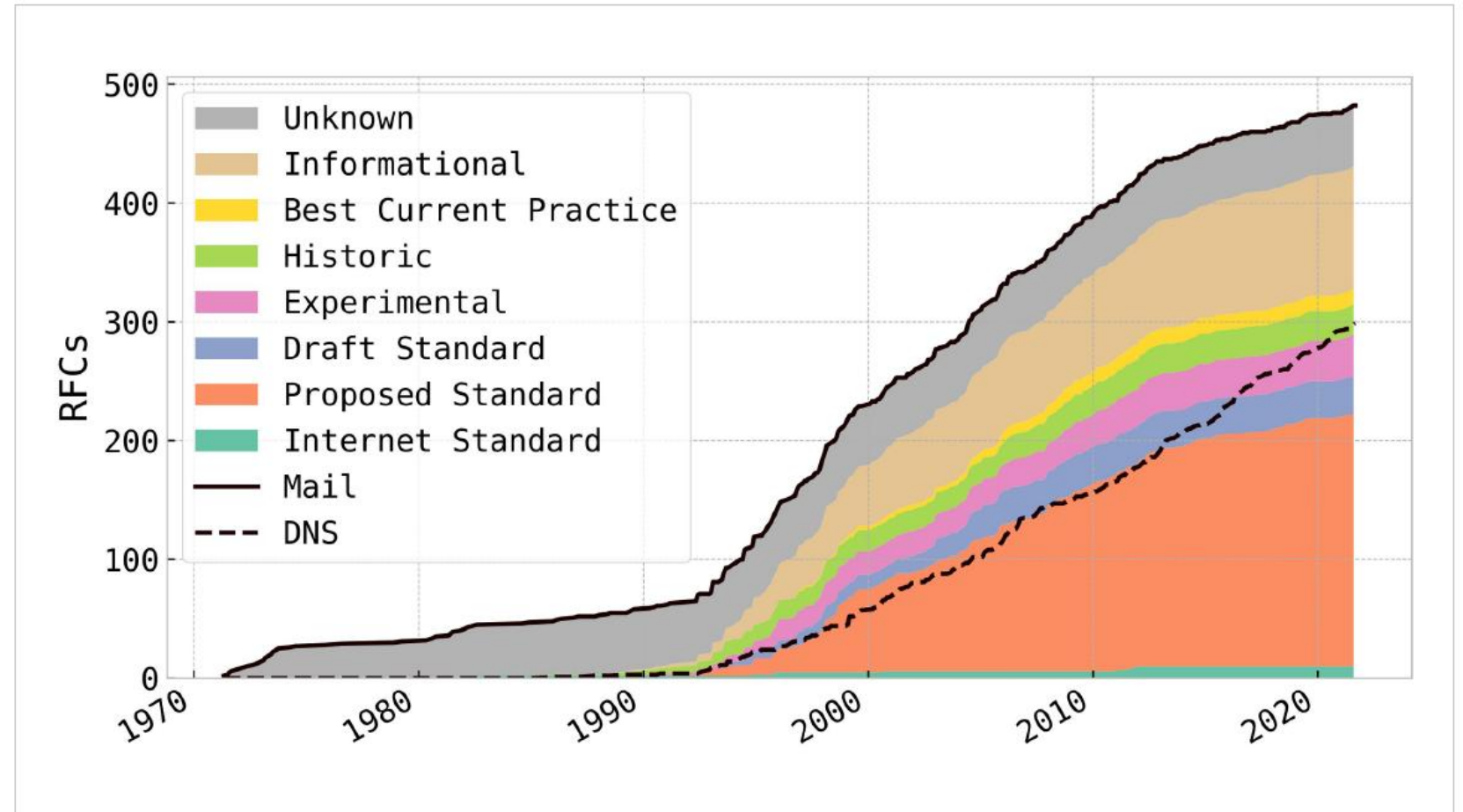
The root of trust is the Key Signing Key (KSK) DNSKEY for the DNS root.
This key is universally known and published.



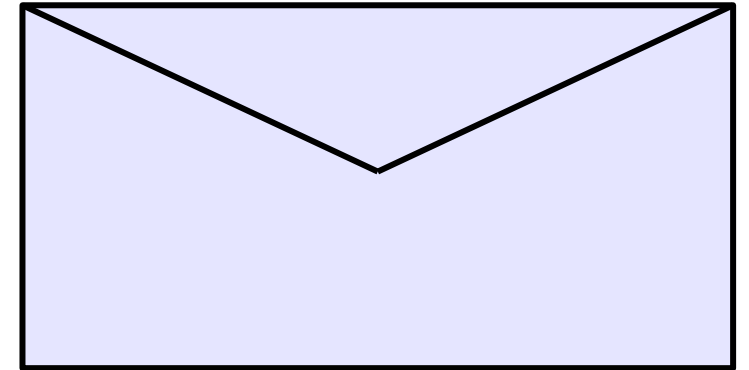
Email

Email-related RFCs

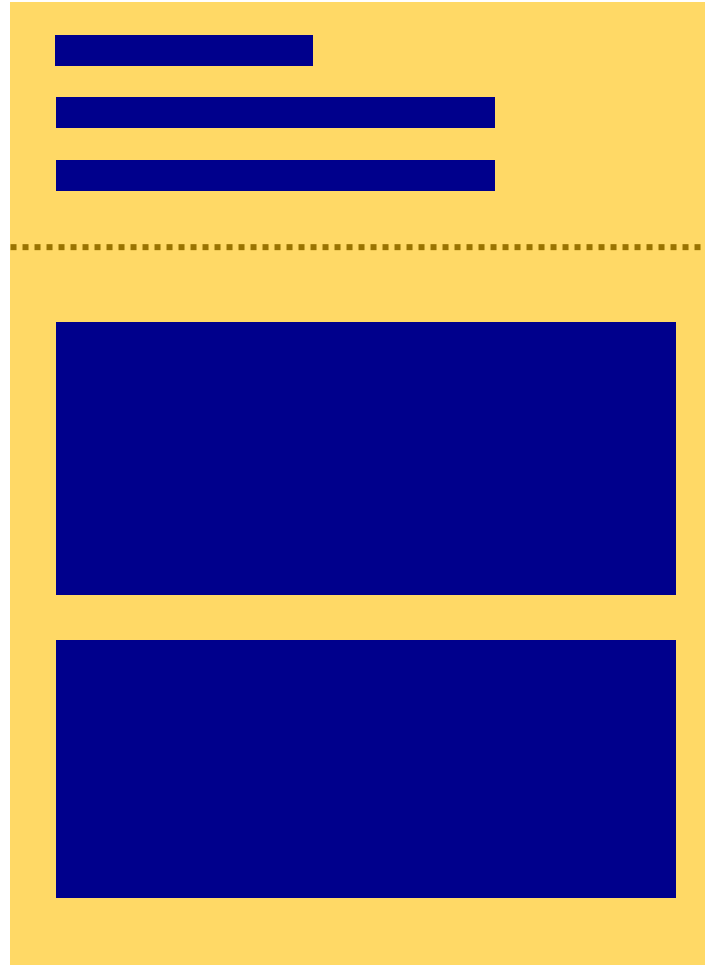
[Holzbauer et al., USENIX ATC 2022]



Message construction: An email consists of envelope including the letter and the actual message content



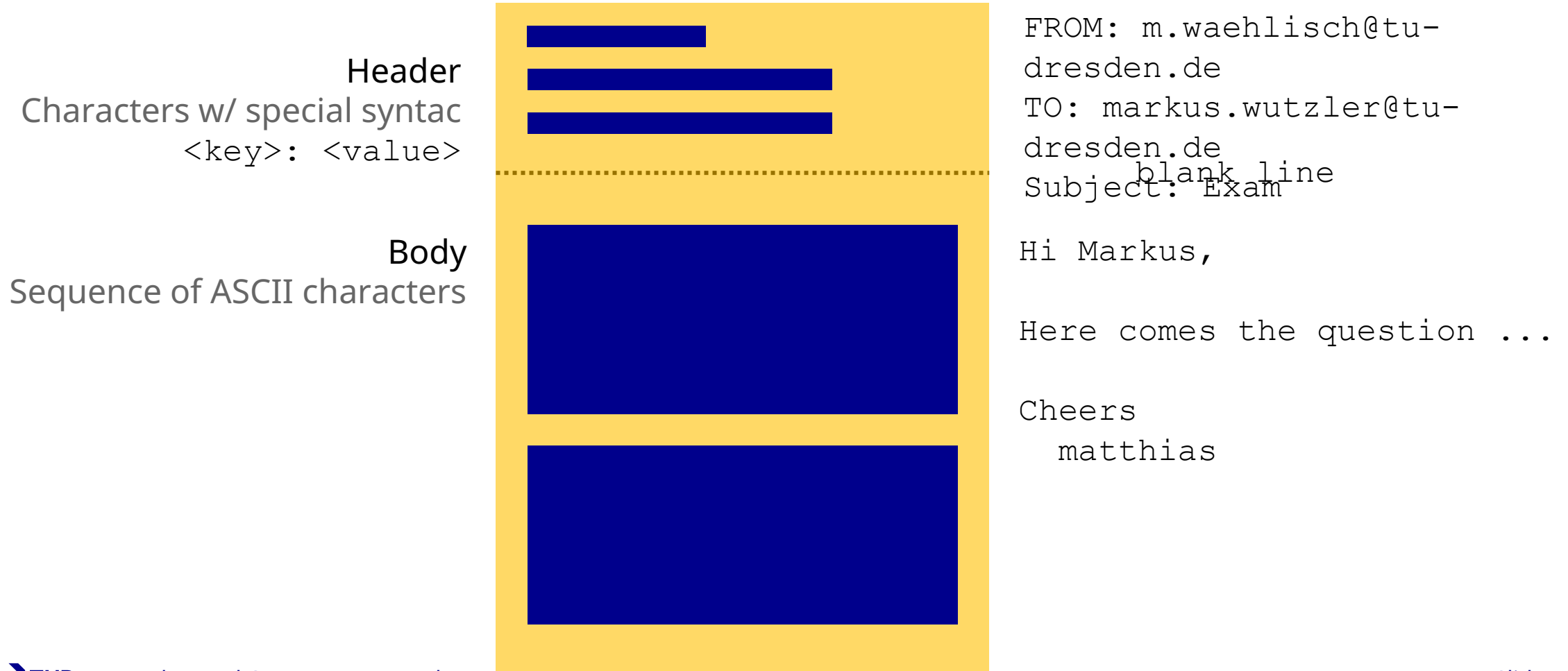
Message content consists of Header Fields and Body



Header
e.g., from, to, bcc

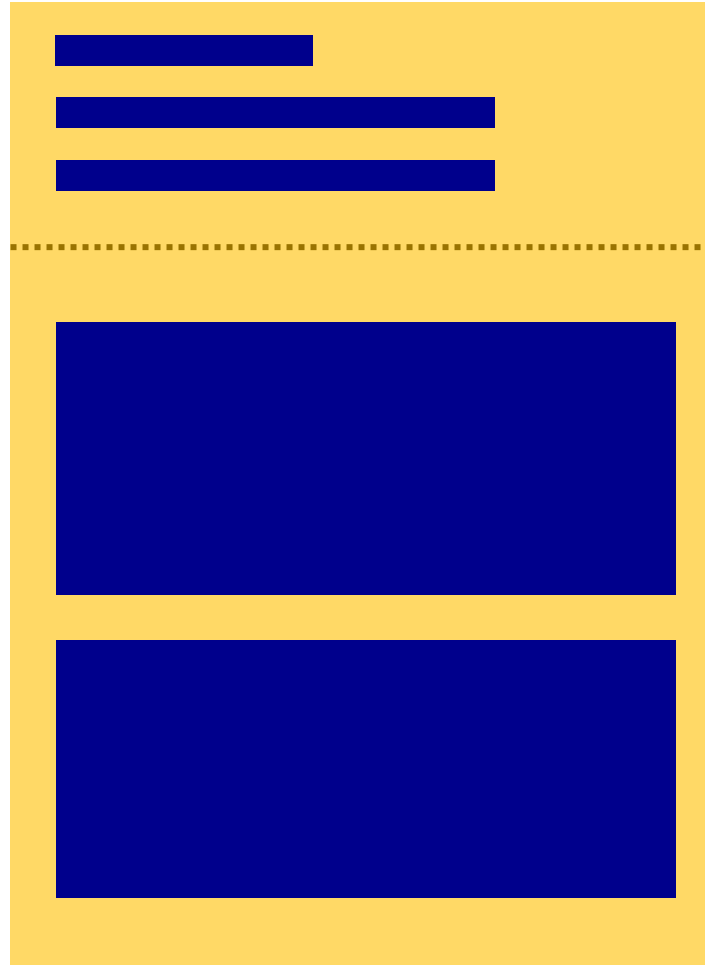
Body
Arbitrary sequence of characters

Message content consists of Header Fields and Body



Challenge: Message allows only 7-bit U.S. ASCII

Use MIME



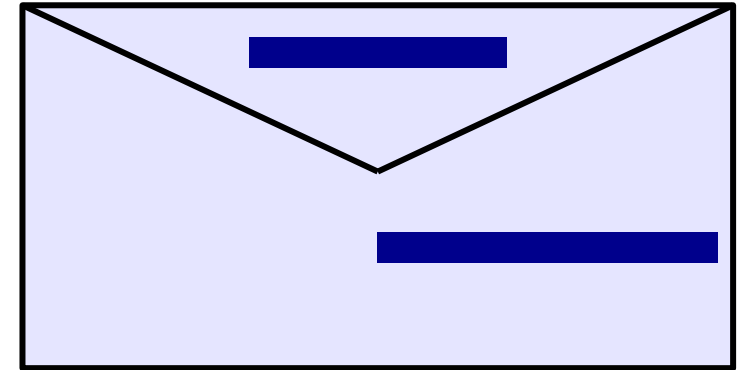
How do you handle

- Special characters, e.g., Umlaut?
- Binary content, e.g., audio, video, images?

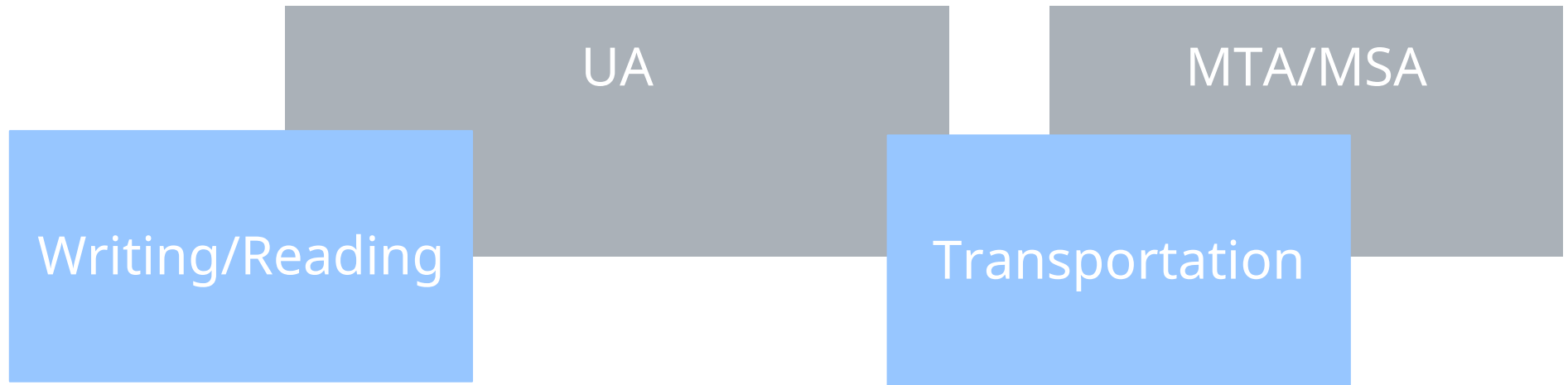
The envelope is created by the MTA or MSA and includes all information for transporting the message

Transport Data (e.g., Env-to)

Some information is redundant to the message header. Reason?



The architecture consists of two subsystems, User Agent (UA) + Mail Transfer/Submission Agent (MTA/MSA)



High-level overview of email architecture

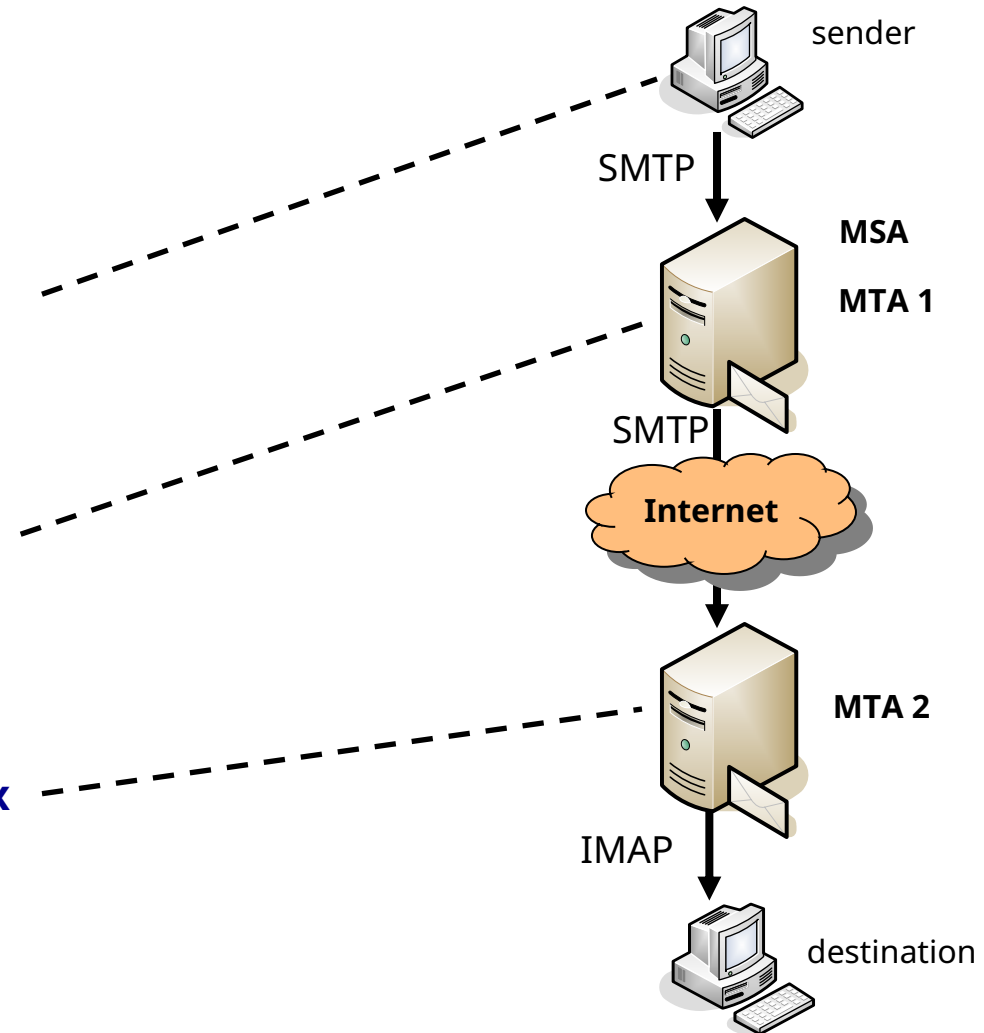
User 1: writes an email

Client 1 (UA 1): formats the email, produces the receiver list, and sends the email to its mail server (MSA of MTA 1, through port 587)

Server 1 (MTA 1): Sets up a connection to the destination's SMTP server (MTA 2) on port 25 and sends a copy of the email

Server 2 (MTA 2): Produces the header of the email and places the email into the destination's mailbox

Users 2: access and read the email, using IMAP, website etc.



SMTP message exchange

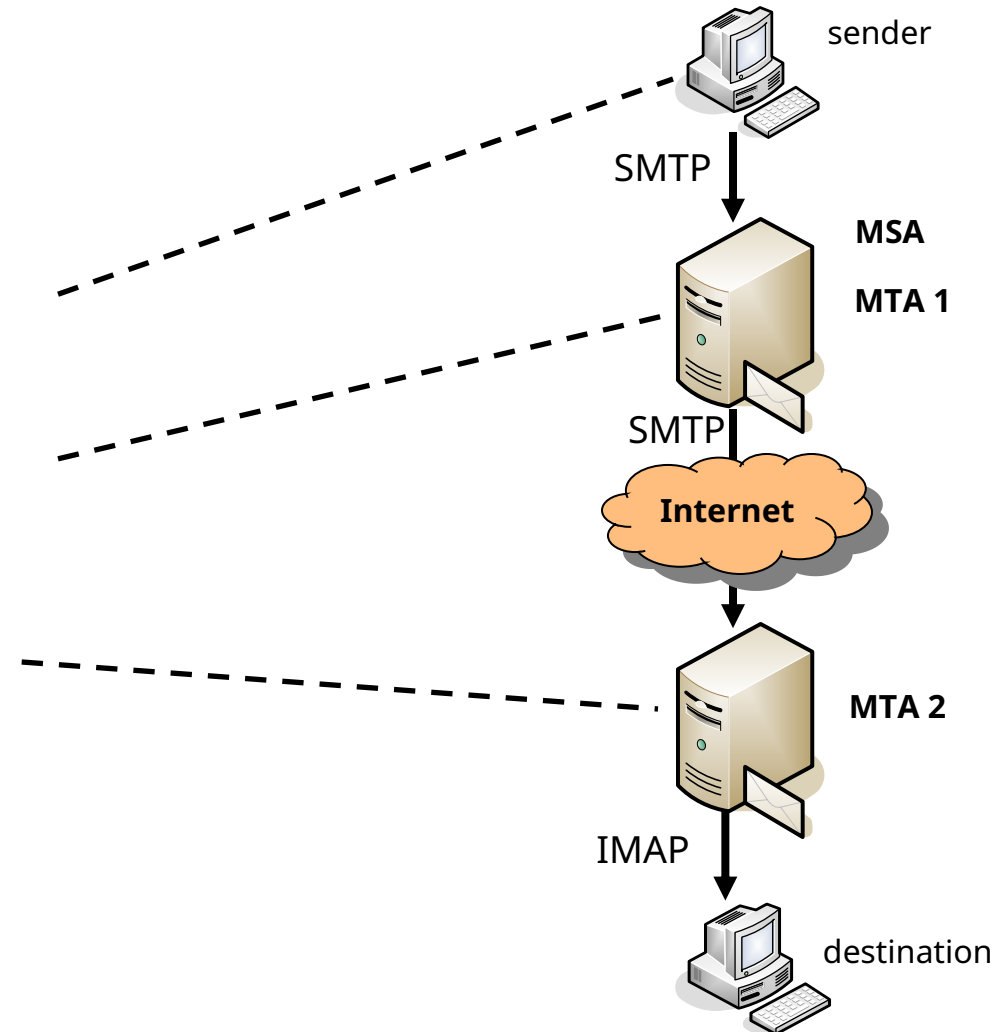
S: 220 mail.a.de SMTP service ready
C: HELO alice.a.de
S: 250 OK
C: MAIL FROM: <alice@a.de>
S: 250 sender ok
C: RCPT TO: <bob@b.de>
S: 250 recipient ok
C: DATA
S: 354 Send mail; end with "." on a line by itself
C: From: alice@a.de
C: To: bob@b.de
C: Subject: testmail
C:
C: Hallo, das ist ein Test...
C: .
S: 250 message accepted
C: QUIT
S: 221 closing connection

Challenges

Sender may send unencrypted email

Sender may spoof the source email address

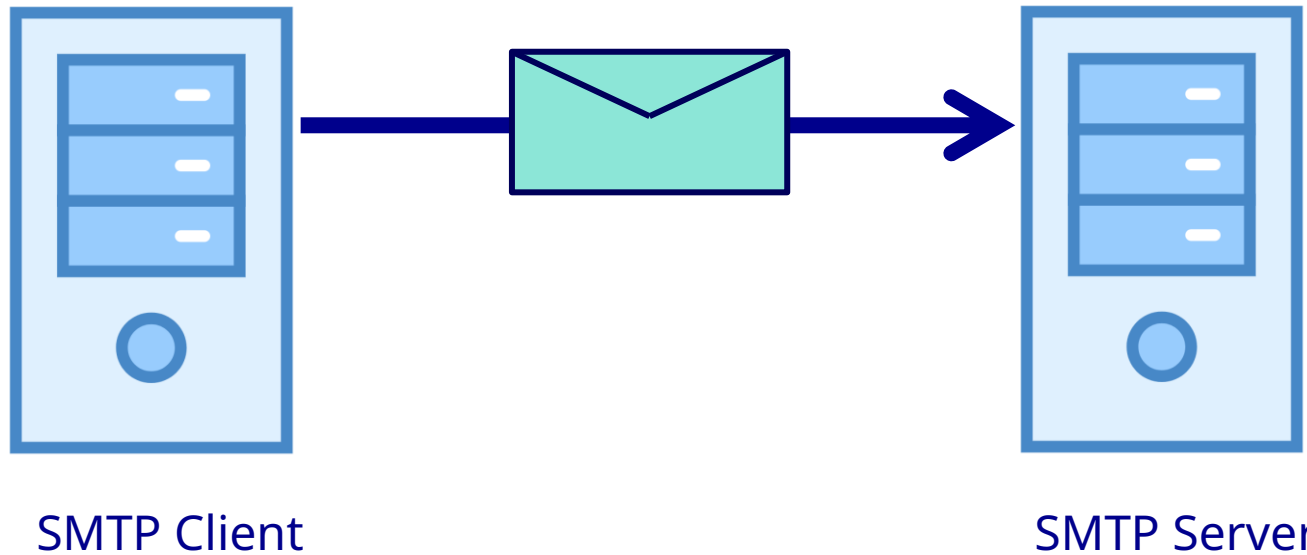
SMTP is not encrypted



Email spoofing, the reason why spam works

Who knows that the source email address is correct?

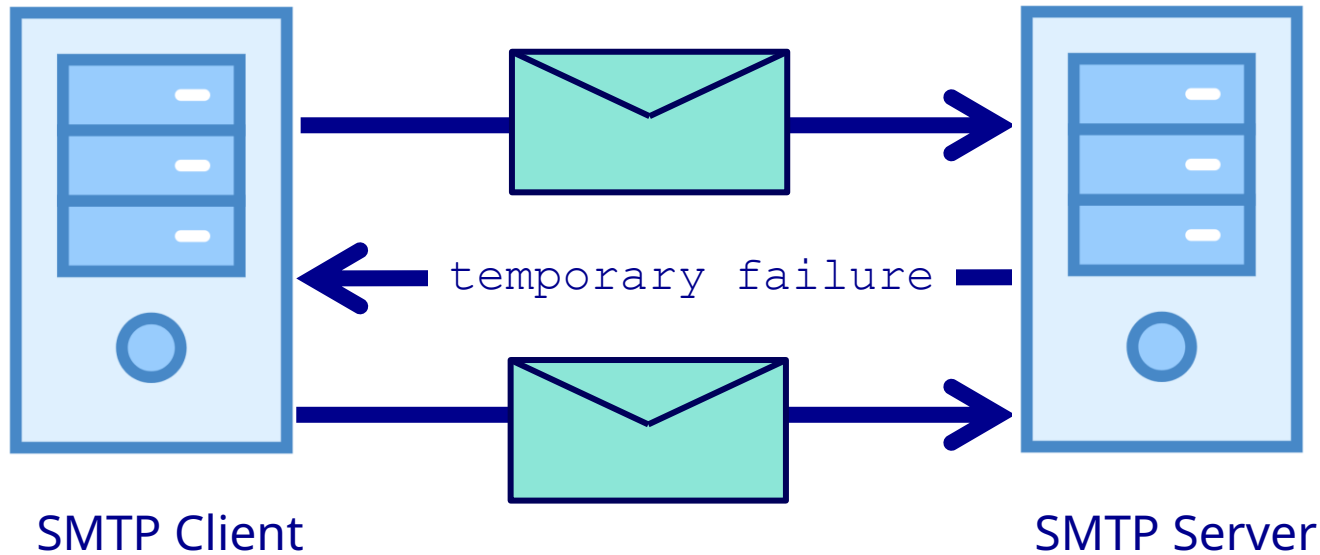
Mitigate email spoofing using greylisting (RFC 6647)



**Did I receive an email
before from this client?**

**No, then respond with 4xx
temporary failure.**

Mitigate email spoofing using greylisting (RFC 6647)



Did I receive an email before from this client?

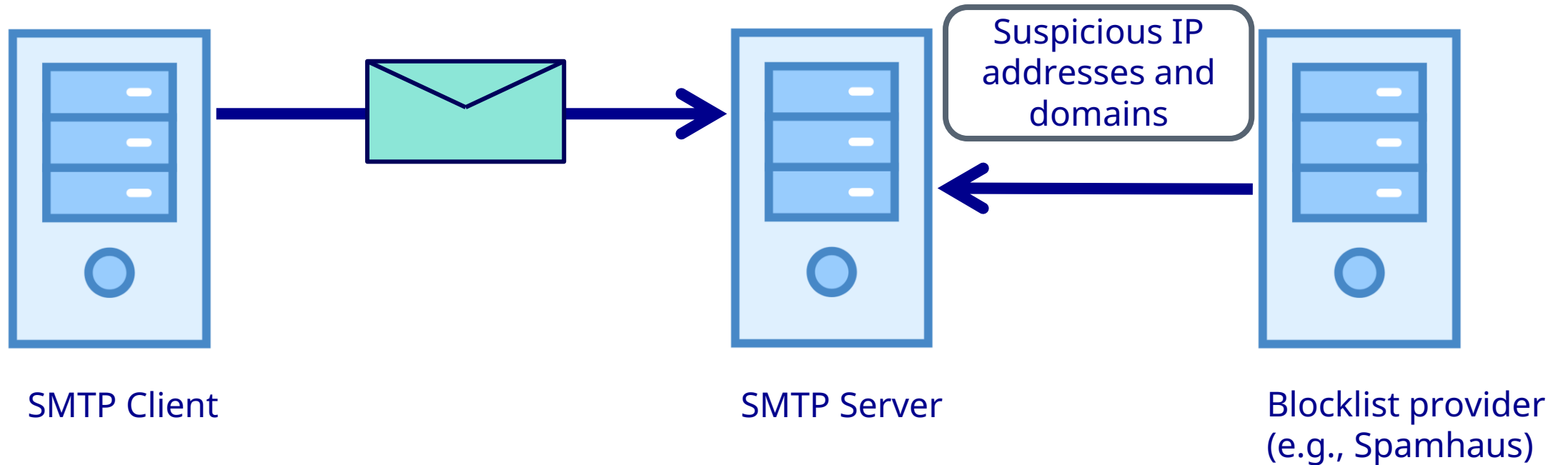
No, then respond with 4xx temporary failure.

Greylisting introduces delays, and email delivery fails if an implementation does not attempt redelivery.

Mitigate email spoofing using greylisting (RFC 6647)



Mitigate email spoofing using blocklisting



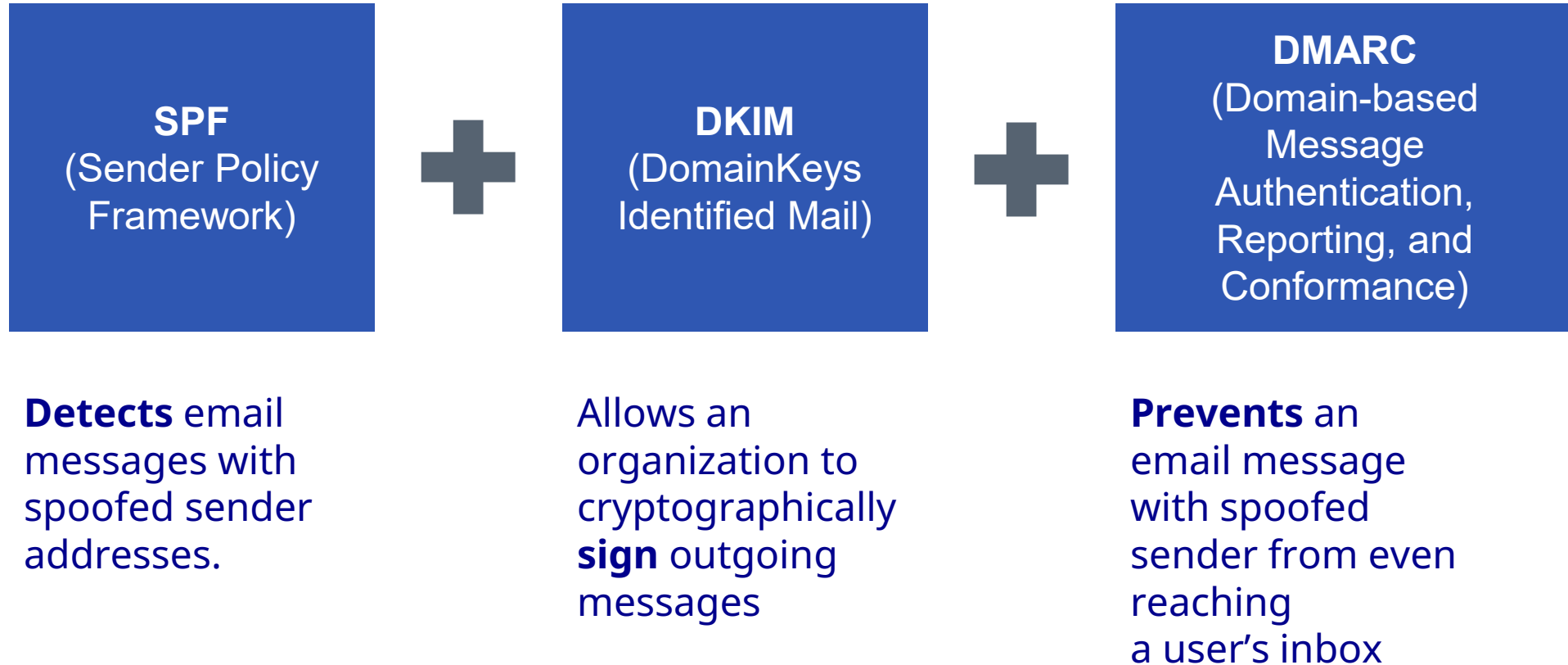
Mitigate email spoofing using blocklisting



Whether your IP prefix is part of a blocklist depends on your email behavior.

A single source may spoil the full prefix.

Mitigate email spoofing Beyond greylisting



SPF: Sender Policy Framework (RFC 7208)

An SPF policy is used to designate the systems that are authorized to send email on behalf of a domain.

A policy is a textual string consisting of various mechanisms, each of which is either:

1. a literal block of one or more IP addresses
2. a hint that can be translated to one or more IP addresses using the Domain Name System (DNS)
3. a recursive SPF policy to replace or extend the current policy

The collection of IP addresses yielded by the policy constitute those explicitly authorized to send mail for a domain.

SPF: Sender Policy Framework (RFC 7208)

Example:

```
foo.com: v=spf1 ip4:192.0.2.1 a:bar.foo.com  
include:foo.net -all
```

Where to publish? DNS!

An SPF record is a DNS record that declares which hosts are authorized to send emails on behalf of a domain.

You use the TXT record for the sender domain.

When an receiving MTA receives an email from user@foo.com, it looks up the SPF policy for foo.com via a DNS query for foo.com type TXT.

DKIM: DomainKeys Identified Mail (RFC 6376)

Idea:

The sending MTA inserts a cryptographic signature which can only be validated with the public key associated with the sender domain

DKIM signature is included in a header in the email message

Where to publish the public key to validate signature? DNS!

Special record of type TXT. DKIM record names follow this format:

```
[selector]._domainkey.[domain]
```

The DKIM header includes a reference to the domain name where that TXT record can be found—a subdomain of the sender domain.

DMARC: Domain-based Message Authentication, Reporting, and Conformance (RFC 7489)

DMARC requires that either SPF or DKIM pass.

It gives a sender domain the ability to specify what an MTA should do with email that fails both. A DMARC policy also designates an email address to which violations should be sent.

Where to publish DMARC policy? DNS!

Special record of type TXT. DKIM record names follow this format:

```
_dmarc.[domain]
```

Summary

Operating a distributed, multi-stakeholder system securely is challenging

DNS is base even for security mechanisms but it needs to be secure before.